Badr University in Assiut (BUA)

Lecture Notes — Week 2

Multi-Layer Perceptrons

Prof. Abdel-Rahman Hedar

Spring 2026

Learning Objectives

By the end of this lecture, you will be able to:

- Explain why multi-layer networks overcome the limitations of the single-layer perceptron
- Describe the MLP architecture and key terminology
- Perform forward propagation step-by-step using matrix notation
- State and interpret the Universal Approximation Theorem
- Design appropriate MLP architectures for classification and regression tasks
- Track tensor dimensions through all layers of an MLP

Contents

1 Recap: Limitations of Single-Layer Networks **2**
1.1 The Perceptron Revisited . . . 2
1.2 Why XOR Is Impossible for a Single Layer . . . 2

2 Multi-Layer Perceptron Architecture **2**
2.1 The Key Insight: Feature Transformation . . . 2
2.2 MLP Terminology . . . 2
2.3 Diagram: 2-Hidden-Layer MLP . . . 3

3 Forward Propagation **3**
3.1 Mathematical Notation . . . 3
3.2 Layer Equations . . . 3
3.3 Step-by-Step Example: 2-Layer Network . . . 4
3.4 Forward Propagation Algorithm . . . 4

4 Why Multi-Layer Networks Work **4**
4.1 Solving XOR: A Worked Example . . . 4
4.2 Hierarchical Feature Learning . . . 5

5 Universal Approximation Theorem **5**
5.1 What the Theorem Says (and Does Not Say) . . . 5
5.2 Why Go Deeper? . . . 5

6 Network Architecture Design **6**
6.1 Design Decisions . . . 6
6.2 Common Architecture Patterns . . . 6
6.3 Parameter Counting . . . 7

7 Summary **7**
7.1 Essential Formulas . . . 7
7.2 Preview: Lecture 3 . . . 8

1 Recap: Limitations of Single-Layer Networks

1.1 The Perceptron Revisited

A single perceptron computes:

$$y = \sigma\big(\mathbf{w}^T\mathbf{x} + b\big)$$

While powerful for linearly separable problems (AND, OR, NAND gates), it *fundamentally cannot* solve problems that require a non-linear decision boundary.

1.2 Why XOR Is Impossible for a Single Layer

The XOR function is the canonical counterexample. The four input patterns $(0,0), (0,1), (1,0), (1,1)$ map to labels $0, 1, 1, 0$ respectively. The positive class $\{(0,1), (1,0)\}$ and negative class $\{(0,0), (1,1)\}$ form two diagonally opposite pairs — no single straight line can separate them.

Perceptron Convergence Theorem

A single-layer perceptron is **guaranteed to converge** if and only if the data is linearly separable. For non-linearly separable data (such as XOR), the algorithm will oscillate indefinitely without ever finding a solution.

2 Multi-Layer Perceptron Architecture

2.1 The Key Insight: Feature Transformation

The crucial observation is that hidden layers act as *learned feature extractors*. They transform the original input space into a new representation in which the (possibly non-linear) classification boundary becomes *linear*. This is sometimes described as "the network learns its own features."

Core Principle

Each hidden layer constructs a new **intermediate representation** of the data. With non-linear activations, successive layers can represent progressively more complex and abstract features.

2.2 MLP Terminology

Input Layer Receives the raw feature vector $\mathbf{x} \in \mathbb{R}^{n_0}$. Its "neurons" are simply the input values — no computation is performed.

Hidden Layer(s) One or more intermediate layers $\mathbf{h}^{(1)}, \mathbf{h}^{(2)}, \ldots$ that transform the representation. The *depth* of a network is the number of layers *excluding* the input.

Output Layer Produces the final prediction $\hat{\mathbf{y}}$. The activation choice depends on the task (sigmoid for binary classification, softmax for multi-class, linear for regression).

Width Number of neurons in a given layer.

Depth Total number of learnable layers (hidden + output).

2.3 Diagram: 2-Hidden-Layer MLP

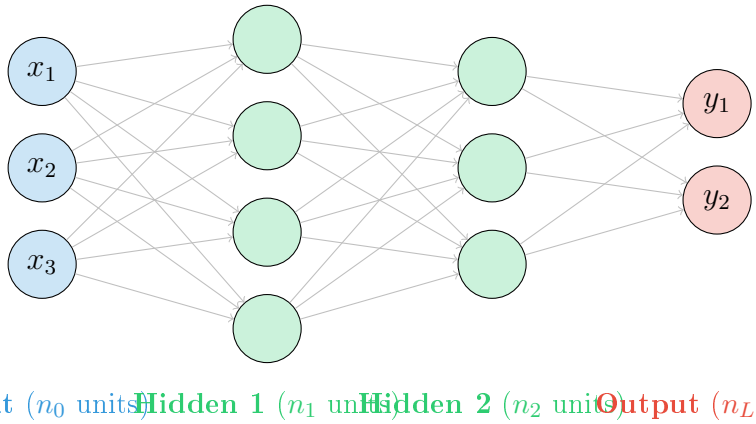


Figure 1: A fully-connected MLP with two hidden layers. Every neuron connects to every neuron in the adjacent layer.

3 Forward Propagation

3.1 Mathematical Notation

For a network with L total layers (indexed $\ell = 1, \ldots, L$), the following notation is used:

Symbol	Meaning	Dimensions
$\mathbf{h}^{(0)} = \mathbf{x}$	Input (layer 0)	$\mathbb{R}^{n_0}$
$\mathbf{W}^{(\ell)}$	Weight matrix, layer ℓ	$\mathbb{R}^{n_\ell \times n_{\ell-1}}$
$\mathbf{b}^{(\ell)}$	Bias vector, layer ℓ	$\mathbb{R}^{n_\ell}$
$\mathbf{z}^{(\ell)}$	Pre-activation at layer ℓ	$\mathbb{R}^{n_\ell}$
$\mathbf{h}^{(\ell)}$	Activations at layer ℓ	$\mathbb{R}^{n_\ell}$
n_ℓ	Number of neurons in layer ℓ	—

Table 1: Standard MLP notation used throughout the course.

3.2 Layer Equations

For each layer $\ell = 1, \ldots, L$:

Forward Propagation Equations

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\,\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)} \quad \text{(pre-activation / linear step)}$$

$$\mathbf{h}^{(\ell)} = \sigma^{(\ell)}\big(\mathbf{z}^{(\ell)}\big) \quad \text{(activation step)}$$

The entire network can be written compactly as:

$$\hat{\mathbf{y}} = f_L \circ f_{L-1} \circ \cdots \circ f_1(\mathbf{x}) \quad \text{where } f_\ell(\mathbf{h}) = \sigma^{(\ell)}(\mathbf{W}^{(\ell)}\mathbf{h} + \mathbf{b}^{(\ell)})$$

3.3 Step-by-Step Example: 2-Layer Network

Forward Pass — 2 inputs, 2 hidden units, 1 output; ReLU activations

Given: $\mathbf{x} = [0.5,\ 0.8]^T$, $\mathbf{W}^{(1)} = \begin{bmatrix} 0.2 & 0.5 \\ -0.3 & 0.4 \end{bmatrix}$, $\mathbf{b}^{(1)} = [0.1,\ -0.2]^T$, $\mathbf{w}^{(2)} = [0.6,\ -0.7]^T$, $b^{(2)} = 0.3$

Hidden layer:

$$\mathbf{z}^{(1)} = \begin{bmatrix} 0.2 & 0.5 \\ -0.3 & 0.4 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix} = \begin{bmatrix} 0.10 + 0.40 + 0.10 \\ -0.15 + 0.32 - 0.20 \end{bmatrix} = \begin{bmatrix} 0.60 \\ -0.03 \end{bmatrix}$$

$$\mathbf{h}^{(1)} = \mathrm{ReLU}\big(\mathbf{z}^{(1)}\big) = \begin{bmatrix} 0.60 \\ 0.00 \end{bmatrix}$$

Output layer:

$$z^{(2)} = 0.6(0.60) + (-0.7)(0.00) + 0.3 = 0.66, \qquad \hat{y} = \mathrm{ReLU}(0.66) = 0.66$$

3.4 Forward Propagation Algorithm

Algorithm 1 Forward Propagation

1: Set $\mathbf{h}^{(0)} \leftarrow \mathbf{x}$
2: **for** $\ell = 1$ to L **do**
3: $\quad \mathbf{z}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}$
4: $\quad \mathbf{h}^{(\ell)} \leftarrow \sigma^{(\ell)}(\mathbf{z}^{(\ell)})$
5: **end for**
6: **return** $\hat{\mathbf{y}} = \mathbf{h}^{(L)}$

4 Why Multi-Layer Networks Work

4.1 Solving XOR: A Worked Example

Consider the two-layer network for XOR with step activation:

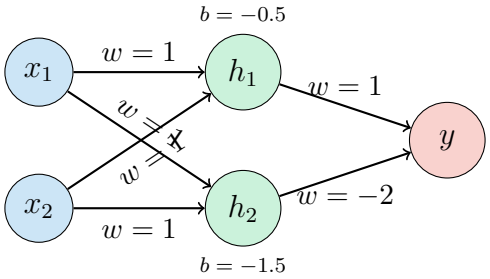


Figure 2: A minimal 2-layer network that solves XOR exactly.

x_1	x_2	$z_1 = x_1 + x_2 - 0.5$	$z_2 = x_1 + x_2 - 1.5$	$h_1 = \text{step}(z_1)$	$h_2 = \text{step}(z_2)$
0	0	-0.5	-1.5	0	0
0	1	$+0.5$	-0.5	1	0
1	0	$+0.5$	-0.5	1	0
1	1	$+1.5$	$+0.5$	1	1

Table 2: Intermediate and final values for the XOR network ($y = \text{step}(h_1 - 2h_2)$).

For $(x_1, x_2) = (1, 1)$: $y = \text{step}(1 - 2 \cdot 1) = \text{step}(-1) = 0$ ✓

4.2 Hierarchical Feature Learning

A key reason deep networks succeed in practice is *hierarchical representation learning*. Early layers detect simple low-level features; later layers combine these into increasingly abstract, task-relevant representations. In image recognition:

- **Layer 1** — low-level edges (horizontal, vertical, diagonal)
- **Layer 2** — mid-level parts (eye shapes, nose contours)
- **Layer 3** — high-level objects (complete faces, expressions)
- **Output layer** — class label (identity or emotion)

5 Universal Approximation Theorem

Universal Approximation Theorem (Cybenko 1989; Hornik et al. 1989)

A feedforward network with a **single** hidden layer containing a *finite* number of neurons with a non-linear activation function (sigmoid, tanh, ReLU, ...) can approximate any continuous function $f : \mathbb{R}^n \to \mathbb{R}^m$ to **arbitrary accuracy** on a compact subset of $\mathbb{R}^n$.

5.1 What the Theorem Says (and Does Not Say)

Interpretation

What it guarantees	What it does NOT guarantee
One hidden layer is sufficient in principle	The number of neurons may need to be *exponentially* large
Any continuous function can be approximated	It does not tell us how to *find* the weights
The approximation can be made arbitrarily precise	It says nothing about generalisation or efficiency

5.2 Why Go Deeper?

Although one hidden layer is theoretically universal, *deep* networks offer substantial practical advantages:

Shallow (1 hidden layer)	Deep (many layers)
May require $O(2^n)$ neurons	Requires only $O(n \log n)$ neurons
Hard to optimise	Better-conditioned optimisation landscape
Poor generalisation	Better inductive biases; stronger regularisation
Example: parity on n bits needs $O(2^n)$ units in 1 layer vs. $O(n)$ in $O(\log n)$ layers.	

Modern Practice

Today's state-of-the-art networks are *very* deep: ResNet-152 has 152 layers; GPT-4 has 96+ transformer layers. Depth, combined with modern training techniques, is the key driver of performance.

6 Network Architecture Design

6.1 Design Decisions

When building an MLP, three main decisions must be made:

1. Depth (Number of Layers)

Start with 1–2 hidden layers for tabular data; increase if the network *underfits*. Typical deep networks for images/text use 5–100+ layers.

2. Width (Neurons per Layer)

A common heuristic: start at 2–3× the input dimension and taper down. Example: $100 \to 64 \to 32 \to 10$.

3. Output Activation

Task	Output activation
Binary classification	Sigmoid ($\to$ probability in $(0, 1)$)
Multi-class classification	Softmax ($\to$ probability distribution over k classes)
Regression	Linear (no activation)

6.2 Common Architecture Patterns

Funnel (Decreasing width): $784 \to 256 \to 128 \to 64 \to 10$. Most common for classification. Progressively compresses the feature representation toward the class space.

Constant Width: $100 \to 128 \to 128 \to 128 \to 1$. Simpler to design and tune; consistent capacity per layer. Common for regression.

Bottleneck (Autoencoder): $784 \to 256 \to 64 \to 256 \to 784$. Forces the network to learn a compact *latent code*; used in dimensionality reduction and generative models.

Inverted Pyramid (Expanding): $10 \to 32 \to 64 \to 128 \to 1000$. Expands from a small latent space; used in decoder architectures and generation tasks.

6.3 Parameter Counting

The total number of learnable parameters for a dense (fully connected) layer is:

$$\#\text{params in layer } \ell = n_\ell \times n_{\ell-1} + n_\ell = n_\ell(n_{\ell-1}+1)$$

MNIST Classifier: $784 \to 128 \to 64 \to 10$

Layer 1:	$784 \times 128 + 128 = 100{,}480$
Layer 2:	$128 \times 64 + 64 = 8{,}256$
Output:	$64 \times 10 + 10 = 650$
Total:	109,386 parameters

7 Summary

Key Takeaways — Lecture 2

1. **Limitation of single-layer networks.** Can only solve linearly separable problems; cannot solve XOR.
2. **Multi-layer structure.** Hidden layers create new feature representations through non-linear transformations, enabling the output layer to apply a linear boundary in the transformed space.
3. **XOR solution.** A 2-layer step-activation network solves XOR exactly. h_1 detects OR; h_2 detects AND; output computes h_1 AND NOT h_2.
4. **Universal Approximation Theorem.** One hidden layer is theoretically sufficient; but depth provides exponential efficiency gains in practice.
5. **Architecture design.** Choose depth, width, and output activation based on task type. Count parameters: $n_\ell(n_{\ell-1}+1)$ per layer.
6. **Forward propagation.** $\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}$; $\mathbf{h}^{(\ell)} = \sigma^{(\ell)}(\mathbf{z}^{(\ell)})$.

7.1 Essential Formulas

Formula	Description
$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}$	Pre-activation (linear step)
$\mathbf{h}^{(\ell)} = \sigma^{(\ell)}(\mathbf{z}^{(\ell)})$	Activation step
$\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$	Weight matrix dimensions
$\hat{\mathbf{y}} = f_L \circ \cdots \circ f_1(\mathbf{x})$	Network as function composition
$n_\ell(n_{\ell-1}+1)$	Parameters per layer

7.2 Preview: Lecture 3

The next lecture introduces **backpropagation**: the algorithm that makes it possible to *train* multi-layer networks by efficiently computing gradients of the loss with respect to all parameters using the chain rule.

Reading Assignment

- Géron — Chapter 10: Introduction to ANNs with Keras
- Buduma — Chapter 2: Training Feed-Forward Neural Networks